

Soft computing for intelligent knowledge-based systems

J F Baldwin, T P Martin and B Azvine

Knowledge-based systems are founded on the idea that knowledge should be declarative, so that it can be easily read, understood, and altered by a human user as well as by a machine. Logic fulfils these criteria, and logic programming has been widely used for implementing knowledge-based systems. One major shortcoming of logic programming is the lack of a mechanism to deal with the uncertainty inherent in many knowledge-based systems. Soft computing is a key technology for the management of uncertainty, although so far its major successes have been centred on fuzzy control rather than higher level information management. This paper outlines some of the issues related to the area of soft computing in knowledge-based systems, and suggests some simple problems to test the capabilities of software. Fril is discussed as an implementation language for knowledge-based systems involving uncertainty, and some of its applications are outlined.

1. Introduction

Soft computing encompasses a range of techniques, generally grouped together as fuzzy logic, neural network theory and probabilistic reasoning — the distinguishing feature being a tolerance of imprecision, uncertainty and partial truth [1]. In commercial terms, the major successes of soft computing so far have been in fuzzy control and to a lesser extent in classification tasks using neural nets.

Soft computing is promoted as a key technology in the information age, providing tools capable of containing the information explosion at a manageable level for humans. Raw data is often not in a form which a human can easily understand. Large volumes of data from fields such as medicine, science and engineering, finance and economics, marketing, social welfare, education, advertising, law, the arts, etc, are available. We store it in large databases, hoping that it will be useful. To be useful we must be able to discover relationships between the attributes of the database, and find rules to predict values of one attribute when others are known. A database may not have all its entries correct, there may be errors, or values may not be known exactly. We require a human/computer partnership where the human is the interested party and the computer has the appropriate software to assist [2]. Fuzzy logic needs to move on from relatively low-level control tasks into this field of intelligent assistance in information processing [3—5]. For example, Dubois, Prade and Yager [6] identify information clarification, information retrieval, and information exploitation as areas in which fuzzy methods can be used. The general area of soft computing is applicable to these problems.

Typically in soft computing, software is developed as a research solution for a specific problem or group of

problems. The complementary nature of the techniques within soft computing often makes it difficult to compare methods. As the use of soft computing increases, we will find more users wanting to take software ‘off the shelf’ and apply it to their own specific problem. Such users may understand the basic theory underlying the software, but should not have to devote a large effort to evaluating and comparing different software packages. There is a need to know the capabilities and limitations of available software. Ideally a library of demonstrator problems will enable an end user to evaluate soft computing tools by finding a problem similar to their own, determine whether or not a particular software package can handle the problem, and also see how easy it is to express the problem using the software. Thus the purpose of a benchmarking suite would be to compare capabilities, with run-time performance very much as a secondary measure.

This paper is divided into four sections, answering the questions identified above:

- an outline of issues related to soft computing in knowledge-based systems,
- some simple problems to test the capabilities of soft computing tools,
- discussion of some available software for creating knowledge-based systems involving uncertainty, focusing on Fril,
- some successful applications of Fril.

2. Uncertainty in knowledge-based systems

At a high level, software can be regarded as a black box taking input data, performing some sort of processing, and returning output data. Uncertainty in one or more of these stages is the key interest — in the input data, in the output data, or in the processing stage. A knowledge-based system imposes constraints on the processing involved — it must be understandable at some level by a human with expertise in the field addressed by the software. That is, the program should be declarative in nature, so that the processing can be understood without reference to the effect of a program on a computer. For many, this requirement leads straight to the use of logic (or a logic-based approach) for knowledge representation; however, it is argued here that pure logic is too restrictive and needs to be augmented with a mechanism for handling uncertainty.

2.1 *The need for uncertainty in knowledge-based systems*

Knowledge-based systems attempt to capture some aspect of human intelligence and reasoning, and almost inevitably involve uncertainty. Two types of uncertainty are important, those depicted either by probability or by fuzzy set theories. Fuzzy set theory is not an alternative to probability theory and the two theories are not competing. These two theories are in fact related by the mass assignment framework [7, 8]. A normalised fuzzy set on X induces a possibility distribution over X which defines a family of probability distributions over X . This family of probability distributions can be represented as a mass assignment which is a probability distribution over the power set of X . Modification to this understanding must be made in the case of un-normalised fuzzy sets, when a degree of inconsistency is present.

A fuzzy set is defined in terms of membership values, but what are these values and how can they be estimated? These are important questions and are equivalent to asking how can a fuzzy set be interpreted. Mass assignment theory uses a voting model to interpret fuzzy sets.

We can appreciate the need for fuzziness even though we do not understand it. Fuzziness is a human phenomenon and has little, if anything, to do with the real world. Humans need to label objects, give description to situations, generalise from one situation to similar ones. If we defined a car precisely we would need to update the definition when a new model came out. A fuzzy understanding of the concept 'car' allows us to appreciate that an object which is similar in most important respects is a car. Fuzziness is necessary for generalisation; most inferences made by people are inductive rather than deductive. Understanding the concept of a car does not mean giving a precise specification of and functionality to each and every part. We choose a set of general features and relations between these features which are important for giving the name 'car' to

several examples. These features and relations will apply to other cases of 'car' and the differences will be lost in the abstractions of the semantics of the features and relations. There will nevertheless be difficult cases for which a decision seems borderline. In these cases we may wish to simply say we cannot make a decision. Fuzzy set theory expects one to be able to give a degree of applicability and therefore have some measure of closeness of match. This assumes there can be degrees of possibility and not just possible and impossible.

Another reason why we use fuzzy labels is one of complexity. We simplify our understanding of the relationship between concepts by using fuzzy labels. We might conclude that 'most tall people wear large shoes'. This description is very economic and easy to handle. We could replace it with numerous more precise statements but would probably find the more complex description too difficult to handle. This is the reason for the success of fuzzy control. A few fuzzy rules give the required breadth of description without losing the accuracy required to make reasonable decisions.

It is accepted that the need for fuzzy set theory arises because our natural language contains words that cannot be given precise definition. Reality needs to be classified, to be divided into parts, and the set of values that a variable can take into broad categories needs to be partitioned. This necessarily introduces fuzzy boundaries and fuzzy sets are used for the classifications.

2.2 *Uncertainty in input and/or output data*

Fuzzy sets are proposed as a way of expressing natural language constructs using uncertain data values. At some level we must be able to say exactly what is meant by a fuzzy set as an item of uncertain data. In a fuzzy controller, it is not necessary for an end user to know anything about the fuzzy sets; however, it should be clear to the designer what membership functions mean. Frequently an 'intuitive' meaning is proposed — e.g. water temperatures can be 'hot' or 'cold', etc. Bezdek [9] suggests that membership functions should be 'adjusted for maximum utility in a given situation'. However, this leaves membership functions with no objective meaning — they are simply parameters to make the software function correctly. For a fuzzy knowledge-based system to be meaningful to a human, the membership functions should have an interpretation which is independent of the machine operation, i.e. one which does not require the software to be executed in order to determine its 'meaning'. Probabilistic representations of uncertain data have a strictly defined interpretation, although eliciting the actual numbers may be difficult. This paper is not concerned with the details of specific approaches — the point is that, in a knowledge-based system, we should be able to determine exactly what is meant by the representation of uncertainty in input or output values.

Even without allowing vagueness, it may be difficult to represent imprecise information such as ‘John’s height is between 1.6 and 1.7 metres’ or ‘Bill’s car is a Porsche or a Jaguar’. The information is expressible in logic; however, computer-based implementations typically assume that information is known completely or not at all. The problem is compounded when one allows vague values — ‘John is of medium height’ or ‘Bill drives an expensive car’.

This form of uncertainty can be modelled using a possibility distribution, either continuous or discrete depending on the underlying domain. However, it is not sufficient merely to extend a programming framework by allowing fuzzy attribute values (or any other uncertain attribute values) — one must also be able to say what is meant by such values, without reference to computer operation; and one must also distinguish between the notions of conjunctive and disjunctive possibility distributions (see also Baldwin, Martin and Pilsworth [10], Dubois and Prade [11], and Yager [12]).

2.3 Uncertainty in the processing stage

Regarding software as a black box, transforming input to outputs, is not altogether satisfactory. A neural net, for example, may perform a complicated transformation of inputs which is not meaningful to the human user, and so is best regarded as a black box. This is a weakness of the neural net approach — how can a user be expected to trust the output of the software if he has no idea what is going on within the system. The degree to which a user needs to (or indeed can) understand the internal workings of a general software package is open to debate; however, some form of transparency is essential for a knowledge-based system, as the ability to explain an answer is a key feature. We must therefore demand a semantics for the processing of the data, in the same way as we demand a semantics for the data — and this should also be understandable without reference to the operations performed by a computer.

3. Logic programming for knowledge-based systems

In the 40-something years of its history as a practical discipline, artificial intelligence (AI) has followed a very different path from mainstream computing. While the early ‘conventional’ high-level languages concentrated largely on numerical computation, AI programmers were more concerned with manipulating symbols, as embodied by the development of LISP. In a sense we have now reached a point of confluence, where many parts of the AI community recognise the need for numerical as well as symbolic processing, and where symbolic computation probably accounts for the majority of computer use through document preparation and dissemination using word processors, Internet, etc. In addition, AI techniques such as expert systems, intelligent help, deductive databases,

grammar, speech and image processing, are becoming part of mainstream computing.

One of the main reasons for AI’s early focus on symbolic computation was the notion that AI programs should be declarative — that is, a program should be open to interpretation as a theory as well as being executable as a series of instructions on a computer. LISP was based on Church’s lambda calculus; in the 1970s, the idea of logic programming [13, 14] became popular — receiving a particular boost when it was chosen as the basis for the Japanese Fifth Generation project in the early 1980s. A pure logic program can be interpreted as a logic theory, or as a sequence of instructions to be executed. Soundness and completeness results can be proved for logic programs, and equivalence of programs can be used to transform code to enhance efficiency without losing ‘accuracy’.

A program clause C_i :

$$A_i \leftarrow B_{i1}, B_{i2}, B_{i3}, \dots B_{in}$$

is regarded both as a logical axiom and as a procedure definition. A query:

$$\leftarrow G_1, G_2, G_3, \dots G_m$$

is regarded both as a logic theorem to be proved from the axioms and as a sequence of procedure calls to be made. An execution step involves unifying a goal G_i with the head of some clause C_j , so that the new goal statement is:

$$\leftarrow (G_1, G_2, \dots G_{i-1}, B_{i1}, B_{i2}, B_{i3}, \dots B_{in}, G_{i+1}, \dots G_m) \Theta$$

where Θ is the unifier for A_j and G_i . Computation terminates when an empty goal statement is produced.

Logic programming deals with axioms whose content is irrelevant — only form is important. However, in order to relate such a program to the real world, one must choose an interpretation, i.e. a mapping between the different symbols in the program and some universe of discourse, U . Each predicate, for example, then defines a relation on U , each constant symbol corresponds to an element in the universe, etc. One can go further and define a set of domains D_i as subsets of the universe of objects U :

$$\forall_i : d_i \subseteq U$$

Domains may be discrete or continuous, and finite or infinite. A predicate then defines a relation, i.e. a subset of the Cartesian product of two or more domains:

$$R_i \subseteq D_{i1} \times D_{i2} \times \dots \times D_{in}$$

where n is the arity of the relation R_i . The relation may be defined by extension, i.e. a set of facts, by intension, i.e.

rules, or by a mixture of both. The extension of all relations is the set of logical consequences of the program.

Logic programming (as represented by Prolog) is well-suited to the implementation of knowledge-based systems. Its declarative leanings facilitate the split into knowledge bases and inference strategy; its well-defined control strategy means that a depth-first inference mechanism is available without any effort, and other inference mechanisms can be programmed straightforwardly; its meta-programming features mean that explanation and more advanced features, such as on-line adaptation, are also relatively easy to program. Of course, logic programming is not the only solution — functional programming also has an elegant mathematical foundation, and procedural languages such as *C* have been used to implement knowledge-based systems. One could argue that since computers interpret binary code, high-level programming languages are unnecessary, and all programs should be written in machine code. However, few people would accept this view since the structures in high-level programming languages enable a convenient representation of algorithms and data. In a similar vein, logic programming provides the most convenient framework for knowledge-based systems — knowledge bases can be represented in a declarative, human readable form and the general programming capabilities allow sophisticated inference strategies and other tools to be written.

3.1 Uncertainty in logic programming

In order to use logic programs to create knowledge-based systems involving uncertainty, the logic programming framework must be extended to express and process uncertainty. The following sections outline the extensions necessary.

3.2 Uncertainty in attribute values

This form of uncertainty can be modelled using a possibility distribution, either continuous or discrete depending on the underlying domain. An attribute value is then either an element of the relevant domain or a fuzzy subset of the domain, and a relation is:

$$R_i \subseteq \{(a_{i1}, a_{i2}, \dots, a_{in}) \mid \\ a_{ij} \in D_{ij} \vee a_{ij} \subseteq_f D_{ij}, 1 \leq j \leq n\}$$

(a_{ij} are attribute values and $\subseteq_f$ denotes fuzzy subset).

The statement, 'the dice score was high', where high represents the fuzzy set $\{4:0.6, 5:0.9, 6:1\}$, should allow us to state that:

the dice score was 4 (with degree C4)
OR the dice score was 5 (with degree C5)
OR the dice score was 6 (with degree C6)

where the term 'degree' is deliberately unspecified, but is a numerically-based value on a scale where 0 represents falsity and 1 represents truth.

Finally it should be noted that the disjunctive interpretation extends the expressiveness of the framework. The conjunctive interpretation 'John speaks French and English and German' is easily restated as the conjunction 'John speaks French' and 'John speaks English' and 'John speaks German'. However, the disjunctive interpretation, 'John speaks exactly one language out of French, English, and German' is more difficult to represent using Horn clauses.

3.3 Uncertainty in queries

The use of uncertain queries is related to the notion of uncertainty in attribute values. Instead of having to define crisp conditions for a database query, e.g. 'find all sales staff who have sold more than 100 units a month for each of the last 6 months', one can use more natural but vague conditions such as 'find all sales staff who have *consistently* sold *large quantities* in the *last few months*' where the italicised terms are defined on the appropriate domains. This reduces the problem of queries cutting out near misses, e.g. the salesman who sells 99 units a month in the example above.

Allowing uncertainty in queries and attributes leads to an inference problem — the uncertain terms need to be matched in some way. There are four cases to examine. Consider a set D of facts and rules, and a query Q .

- Q is a crisp query, D contains complete information. In this case, it is a simple matter to establish whether Q is true or false by determining whether or not it is a logical consequence of D . For a simple example, let D contain the fact that John's height is 6 ft and Q_1 be the statement 'John's height is 6 ft' and Q_2 the statement 'John's height is 5 ft'. Clearly Q_1 is true and Q_2 is false. This is the situation handled by classical logic.
- Q is a fuzzy query, i.e. it contains a vague predicate or quantifier, and D contains complete information. In this case, we may not be able to derive Q from D with complete certainty. Using a similar example to that given above, let D contain the fact that John's height is 5 ft 10 in and Q be the statement 'John is tall'. A numerical estimate can be attached to the conclusion Q , as Q cannot be said to be definitely true or false.

An alternative approach is to disallow any fuzzy statements and insist on a crisp definition of predicates ('tall' in this instance). This has the disadvantage that

some terms in the formal language used to represent knowledge may not have the same meaning as the same terms used in everyday language — e.g. ‘tall’ could be defined to be any height over 6 ft.

- Q is a crisp statement, D contains incomplete information. In this case, Q is definitely true or false but it may not be possible to determine which, due to lack of information. For example, let D contain the fact that John’s height is 6 ft or 5 ft 11 in, and Q be the statement ‘John’s height is 6 ft’. The database does not contain sufficient information to establish the truth of Q , but it can give information on the possibility or probability that Q could be true.
- Q is a fuzzy statement, D contains incomplete information, e.g. D contains statements ‘Bill is average height’ ‘John is several centimetres taller than Bill’, and Q is the statement ‘John is tall’. This is the most complex situation and is considered further below.

3.4 Uncertainty in relations

A crisp relation is a subset of the Cartesian product of two or more domains:

$$R_i \subseteq D_{i1} \times D_{i2} \times \dots \times D_{in}$$

where n is the arity of the relation R_i . A logic program computes tuples in a relation, i.e. tuples which satisfy some logical condition. As suggested above, modelling uncertainty in the system forces us to allow for the possibility that tuples do not completely satisfy a predicate — for example, let us take the relation:

$$\text{Fluent-Speaker} \subseteq \text{Person} \times \text{Language}$$

and consider the tuples (John, English), (John, French), (John, Spanish). If John is a native English speaker, and knows no French, then the first two tuples respectively belong and do not belong to the relation. However, if John knows a little Spanish then the final tuple lies between these two extremes. Again, one can attach a numerical value between 0 and 1 to show where the tuple lies in the range between false and true — but it must be clear what this number actually means.

One could argue that there is an equivalence between uncertainty in one attribute value of a tuple in a binary relation and an uncertain monadic relation. For example, the data that John is *young* could be modelled using a binary relation:

$$\text{Age} \subseteq \text{Person} \times N_{130}$$

where N_{130} is the set of natural numbers less than 130. Then (John, *young*) would be a tuple in the relation Age with an appropriate definition for the fuzzy set *young*.

Alternatively, monadic relations, such as *Young*, *MiddleAged*, *Old*, etc, could be used:

$$\text{Young} \subseteq_f \text{Person}, \text{MiddleAged} \subseteq_f \text{Person}, \text{ etc,}$$

where $\subseteq_f$ indicates fuzzy subset-hood and the tuple (John) may have uncertainty as to the degree of membership in *Young*, *MiddleAged*, etc. It is important to know whether these are equivalent representations — particularly for the knowledge engineer when designing the system. It is noted that the binary relation form allows crisp and uncertain data to be stored (e.g. both ‘Mary is 29’ and John is *young*) without loss of data; furthermore, it allows more flexibility in queries (e.g. ‘find people whose age is *around 40*’) with the vague term *about 40* defined at query time.

More generally in a logic program, clauses may be conditional. A numerical value between 0 and 1 indicates that there is uncertainty attached to the rule — again it must be clear what exactly is represented by this uncertainty.

3.5 Uncertainty in inference

Uncertainty in inference has been touched on in the preceding sub-sections. Equality is the first logical concept that must be extended, as it is no longer sufficient to look for simple syntactic equality. In logic programming terms, this means the notion of unification must be extended in some way [15]. Since unification is the fundamental operation of logic programming, this extension is only a realistic alternative if it can be made efficient — see Baldwin, Lawry and Martin [16] for example.

The main problem with adding uncertainty to a logic programming system is determining how to combine the numerical uncertainty associated with clauses used in a proof. Probability is the best defined and longest established numerical treatment of uncertainty, yet still hits problems when combined with logic. The problem can be illustrated with a simple example.

A box contains coloured objects which are either red or blue and either cubes or spheres. It is known that 60% of the objects are red, and that 30% are cubes. Let:

$$p \text{ denote object is red, and } q \text{ denote object is a cube,}$$

then it is known that $Pr(p) = 0.6$ and $Pr(q) = 0.3$, but almost nothing can be said about $Pr(p \wedge q)$.

Assume now that further information is available, namely 60% are red cubes, 15% are red spheres, 12% are blue cubes, 13% are blue spheres:

$$Pr(p \wedge q) = 0.6$$

$$Pr(p \wedge \neg q) = 0.15$$

$$Pr(\neg p \wedge q) = 0.12$$

$$Pr(\neg p \wedge \neg q) = 0.13$$

The next step is to express the uncertain rule that if an object is red, then it is probably a cube. The logic formula $p \supset q$ is true in 85% of cases, but the conditional probability $Pr(q | p) = 0.8$. Does uncertainty attached to a rule have a logical or a statistical basis? To further complicate the situation, a programmer may want to represent a rule obtained from an unreliable expert. In a logic programming environment, it must be ensured that the programmer is aware of the interpretation of uncertainty embodied in the system. Of course, when the numbers are interpreted as *degrees of truth*, it may be easier to retain truth-functionality.

Integrating the uncertainty-handling mechanism into the system also presents a choice. A logic program can be regarded as a split:

$$\text{Program} = \text{Logic} + \text{Control},$$

and we have the possibilities:

$$\text{Uncertain program} = \text{Logic} + \text{Control} + \text{Uncertainty},$$

or

$$\text{Uncertain Program} = \text{Uncertain Logic} + \text{Control}.$$

In other words, either the logical basis of the system can be extended, e.g. Lee [17] defined a fuzzy resolution method, or the calculation of uncertainty can be separated from the inference process. A proof of the conclusion is found; a separate mechanism considers the uncertainty attached to the clauses used in the proof, and calculates the resultant uncertainty. This is the mechanism used in Fril (see section 4).

3.6 Fusion of uncertain conclusions

In logic programming, one is generally content to find a single solution to a query. Given a binary predicate p , the question might be asked 'are there values X and Y such that $p(X, Y)$ holds?' — strictly speaking, a yes/no answer is all that is required, although instantiations for X and Y are also helpful in many cases. Because of the execution strategy used, Prolog systems will by default return answers one at a time (including duplicate answers). This is not adequate for a system where answers may be qualified by uncertainty. It is possible that a particular solution can be derived in more

than one way from the program clauses — each proof path having a different calculation of uncertainty and hence a different final value. If two proof paths lead to the conclusion $p(a, b)$ with uncertainty values c_1 and c_2 , how should these be combined into an overall value? This is referred to as the problem of fusion.

Clearly any fusion function f which combines uncertainty values should satisfy end-point criteria such as:

$$f(\text{True}, \text{True}) = \text{True}, \quad f(\text{False}, \text{False}) = \text{False},$$

and be commutative and associative:

$$f(x, y) = f(y, x) \quad f(x, f(y, z)) = f(f(x, y), z)$$

A related problem arises from treating rules individually. Consider universes $U_Y = \{a, b, c, d, e\}$, $U_X = \{c_1, c_2, c_3\}$ and two rules

$$Y = \{a, b\} \text{ IF } X = c_1$$

$$Y = \{c, d\} \text{ IF } X = c_2$$

with an input $X = \{c_1, c_2\}$

The first rule gives $Y = \{a, b, c, d, e\}$, the second gives $Y = \{a, b, c, d, e\}$, and intersecting gives $Y = \{a, b, c, d, e\}$.

Logically it follows that other rules can be used from which the correct solution can be inferred. From the above example, the following rule is valid:

$$Y = \{a, b, c, d\} \text{ IF } X = \{c_1, c_2\}$$

from which it can be inferred:

$$Y = \{a, b, c, d\}$$

which is more accurate than the inferences from individual rules.

4. Mass assignment theory and Fril

This section outlines the methods used in Fril to cope with the problems discussed above. Fril is founded on the theory of mass assignments, which unites fuzzy and probabilistic uncertainty in a single framework. It is covered in Baldwin [8] and Baldwin, Martin and Pilsworth [10], and only the essential points are summarised here for completeness.

4.1 Essentials of mass assignment theory

Mass assignment theory gives a semantics of fuzzy sets using a voting model involving human voters. Consider a dice and a representative set of people labelled 1 to 10. Each person is asked to accept or reject the dice score of x as

small. They can believe x is a borderline case but they have to make a binary decision to accept or reject. The membership of x in the fuzzy set **small** is taken to be the proportion of persons who accept x as **small**.

Thus it is known that everyone accepted 1 as small, 90% of persons accepted 2 as small and 30% of persons accepted 3 as small. Only the proportions who accepted each score are known and not the complete voting pattern of each person. It is assumed that anyone who accepted x as being small will accept also any score lower than x as being small. With this assumption, the following voting pattern can be stated:

1	2	3	4	5	6	7	8	9	10	persons
1	1	1	1	1	1	1	1	1	1	everyone accepts 1
2	2	2	2	2	2	2	2	2		90% accept 2
3	3	3								30% accept 3

Therefore 1 person accepts {1}, 6 persons accept {1, 2} and 3 persons accept {1, 2, 3} as being the possible sets of scores when told the dice is small. If a member is drawn at random then the probability distribution for the set of scores this person will accept is:

$$\{1\} : 0.1, \{1, 2\} : 0.6, \{1, 2, 3\} : 0.3$$

This is a probability distribution on the power set of dice scores, known as a mass assignment and written as:

$$m_{\text{small}} = \{1\} : 0.1, \{1, 2\} : 0.6, \{1, 2, 3\} : 0.3$$

It is, of course, a random set [18] and also a basic probability assignment of Shafer Dempster theory [19]. Here it is given the name of mass assignment because it is used in a different way to Shafer Dempster. The mass assignment corresponds to a family of distributions on the set of dice scores. Each mass associated with a set of more than one element can be divided in some way among the elements of the set. This leads to a distribution over dice scores and there are an infinite number of ways in which this can be done. The least prejudiced distribution divides the masses among the elements of the set associated with them according to the prior for the dice scores.

The converse of this problem is: given the dice value is **small**, what is the distribution over possible dice scores for the dice value? This is only a small modification from the following non-fuzzy case. If the dice score is even then we would say that the scores 2, 4, and 6 were equally likely for a fair dice. In this case a mass of 1 is put with the set {2, 4, 6} and the mass is split equally among the elements of the set of possible scores. The only difference between this and our fuzzy case is that the mass assignment came from a fuzzy set rather than a crisp set. It also makes sense to ask for the probability that the dice value will be in a certain set

given that the dice value is in some other set. For example the question could be asked: 'what is the probability that the dice value will be {1 or 2} given that it is even':

$$Pr(\{\text{dice value in } \{1, 2\} \mid \text{dice value is even}\}) = 1/3$$

Similarly, the probability of the dice value being **about_2**, when it is known to be **small**, can be sought, where **about_2** is a fuzzy set defined by:

$$\text{about_2} = 1 / 0.4 + 2 / 1 + 3 / 0.4$$

with mass assignment

$$m_{\text{about_2}} = \{2\} : 0.6, \{1, 2, 3\} : 0.4$$

Figure 1 shows the equivalent of this in tabular form. A mass assignment with the least prejudiced distribution can be used to obtain a point value probability. The continuous case can be treated either analytically or by means of a discrete sampling approach.

		small		
		0.1 {1}	0.6 {1, 2}	0.3 {1, 2, 3}
about_2	{2}	f 0.06	u 0.36	u 0.18
	0.4 {1, 2, 3}	t 0.04	t 0.24	t 0.12

$Pr(\text{about_2} \mid \text{small}) = [0.4, 0.94]$

Fig 1 Tabular form of the probability of the dice being **about_2** when it is **small**.

4.2 Fril

Fril extends logic programming (Prolog) by allowing fuzzy sets as fundamental data objects, and associating support pairs with clauses. A support pair is an interval containing a point-value probability. The probabilistic fuzzy rules [8, 20, 21] of Fril represent our theory for the way the variables of our application relate to one another. There is no restriction to using simply flat IF ... THEN rules normally associated with fuzzy control. The rules are logic programming clauses in which the variables associated with predicates can be instantiated to any term, including fuzzy sets, and conditional probabilistic intervals can be associated with each rule. These have various interpretations dependent on the type of rule, but in all cases represent an interval containing the conditional probability of the head of the rule given some instantiation of the rule body.

In addition to the normal logic programming style of execution, Fril allows three types of rules involving uncertainty:

- the basic rule, for example:

```
((suitability X for sports stadium Y is high)
 (access X from parts of city is easy)
 (cost_to_build Y at X is fairly_cheap)) : (0.9 1)
```

which states that there is a high probability (0.9 — 1) that any place *X* is highly suitable to build a sports stadium *Y* if *X* is easily accessed and *Y* can be built fairly cheaply at *X* (italicised words represent fuzzy sets),

- the extended rule, for example:

```
((shoe_size man X is large)
 ( ((height X is tall)
    ((height X is average))
    ((height X is small))) ) : (0.8 1) (0.5 0.6) (0 0.1)
```

which states that the probability of a tall man wearing large shoes is at least 0.8, the probability that an average height man wears large shoes is between 0.5 and 0.6, and the probability that a small man wears large shoes is 0.1 or less — the rule can be thought of as representing the relationship between two variables, *S* (shoe size) and *H* (height of man), where *S* is instantiated to large while *H* has three possible instantiations; the rule expresses $Pr(S \text{ is } \textbf{large} | H \text{ is } h_i)$ where h_i is a particular fuzzy instantiation of *H* (this type of rule is useful to represent fuzzy causal nets and many other types of applications),

- the evidential logic rule, for example:

```
((suitability_as_secretary person X is good)
 (evlog most (
   (readability of X is high) 0.1
   (neatness X is fairly_good) 0.2
   (qualifications X are applicable) 0.1
   (concentration X is long) 0.3
   (typing_skills X is very_good) 0.2
   (shorthand X is adequate) 0.1)) : (1 1)(0 0))
```

which says that a person's suitability as a secretary is good if most of the weighted features in the body of the rule are satisfied — the term **most** is a fuzzy set chosen to provide optimism for those persons who satisfy the criteria well and pessimism for those who satisfy the criteria badly, the italicised words are fuzzy sets defined in the program, the satisfaction of features such as (qualifications *X* are applicable) is determined from another rule, and the weights associated with the features can be optimised if a data base of examples is provided [10, 22].

The three Fril rules are of the form:

< head > IF < body > : < list of support pairs >

The list of support pairs provides intervals containing conditional probabilities of some instantiation of the head, given some instantiation of the body. A generalised Jeffrey's rule for support pairs is the basic inference rule of Fril, and mass assignment theory is used where necessary to combine fuzzy and probabilistic uncertainty. Full details are given in Baldwin [8], and Baldwin, Martin and Pilsworth [10].

4.3 Fuzzy sets in Fril

Fril allows uncertainty in attribute values to be represented by fuzzy sets. The fuzzy sets may be defined on an underlying domain which may in turn have a prior distribution defined over it. Fril allows fuzzy sets to represent uncertain values and extends the unification mechanism to cope with this. Additionally, continuous fuzzy subsets can be interpreted as fuzzy numbers and simple arithmetic performed on them.

4.4 Execution

Logic programs are executed by posing a query, which is answered by determining whether that query (or some instance of it) is a logical consequence of the clauses in the knowledge base. Execution is a search over an and-or tree. Execution of a support logic program is also a search over the and-or tree, with additional computation defined by mass assignment theory to calculate supports associated with each solution. The entire and-or tree is searched so that all proof paths may be taken into account. Fril is implemented as a compiler for the abstract Fril machine [23], which is a modification of the Warren Abstract Machine [24] used in most Prolog compilers. This yields a highly efficient support logic programming system. Clauses (possibly involving supports and fuzzy sets) are compiled into sequences of abstract machine instructions, which can be executed by a software emulator or by a hardware implementation of the abstract machine [10, 16, 25, 26].

5. Applications of Fril

Fril has been applied to a wide range of problems. A few are listed below.

5.1 Fuzzy data browser

In many large databases, information may be incomplete or uncertain. Even where everything is known, the volume of data can obscure important relationships within the data. Rules derived from the data can illuminate relationships, and represent fuzzy data compression since

the database can be approximately reconstructed from the rules.

The Fril data browser can be used to form rules which predict the value of an attribute in one tuple from known values of other attributes in that tuple, plus the attribute values in other 'similar' tuples. The database is partitioned into fuzzy subsets containing similar values of the variable under consideration; each rule then uses fuzzy sets to summarise values of other variables in that partition. In this way, rules which are meaningful to a human expert can be generated. The user can also suggest relationships in the data, and test them — for example, a supermarket manager might like to know how well the statement 'most customers who buy expensive wine often buy exotic cheeses at the same time' matches transaction data for his store. Applications of the data browser include:

- image classification, deriving and applying rules to categorise regions of a partitioned image into classes such as vegetation, road, building, vehicle, etc [27],
- generating rules for hand-written character recognition [28],
- learning rules to classify odours into categories such as unpleasant, strong, very acidic, using signals from conductive polymer sensors [29—31],
- classifying faces as male or female, given a set of 18 measurements referring to features such as thickness of eyebrows, distance from bottom of nose to top of mouth, etc [2],
- predicting values and detecting empirical relationships in a database of measurement from geological experiments [30],
- monitoring data from an aircraft black-box flight recorder, where it is necessary to detect anomalous readings which can either indicate that a piece of equipment is malfunctioning, or that measurements are being reported incorrectly,
- classifying underwater sounds, where the browser must generate rules which can be understood by humans — this application has achieved very high success rates (up to 92%), outstripping the performance of a standard neural net package, yet retaining the transparency of the rule based approach, while the neural net, in contrast, provides no explanation or insight into the classification process [32],
- detecting changes in satellite images, using fuzzy rules derived from examples — the system is intended to monitor an area and discover changes related to building construction or demolition; during testing, the system detected a small change involving re-roofing houses which had previously not been noticed.

5.2 Fuzzy control module

A fuzzy control module has been written in Fril, giving a choice of inference strategies, (support logic, Mamdani, Lukasiewicz, Kleene-Dienes), aggregation operators (intersection, dempster, union) and defuzzification methods (expected value, centroid, maximum membership). Unlike many systems, a hierarchy of rules can be used. Fril is particularly suited to investigating higher-level control problems, where (for example) an expert might know that one control strategy is appropriate in one set of circumstances, but a different strategy is needed in different circumstances. This high-level expert knowledge is easy to model in Fril.

5.3 Intelligent manual

The MUNVAR project funded by the CEC identified a need for an intelligent interface to technical literature. The specific case chosen as a demonstrator was a study [33] by the Swedish Nuclear Inspectorate into techniques of assessing the safety of nuclear waste repositories. This study documented methods that could be used, and analysed their validity at a hypothetical site. The document is intended to give guidance to waste management companies preparing a safety assessment for a specific site.

Initial objectives were that the system should be able to:

- trace back from a conclusion to the information justifying that conclusion,
- find the modelling approach used for a given sub-problem,
- aid in examining the consequences of changing the modelling approach,
- handle the uncertainty inherent in different modelling approaches, ranging from uncertainties in experimentally determined values through to determining whether a particular modelling approach is valid.

Subsequent development of this work has focused on a loosely coupled package involving Fril which handles knowledge-based aspects and Mathematica which deals with mathematical modelling. The system has been applied to target motion analysis (UK Defence Research Agency) and management of uncertain engineering data (Intera Ltd).

5.4 Aircrew modelling

The Defence and Avionics Division of Cray Systems Ltd used Fril to produce a modelling tool for representing the behaviour of aircrew in helicopter and fixed wing operations [34]. The tool is used to produce models of aircrew behaviour that can run within a simulation

environment to aid the assessment of pilot workload on specific missions and examine the workload effects of various avionics and decision support aids in the cockpit. It is also to be used for modelling aircraft in combat simulations.

The model represents the three fundamental tasks of the aircrew — data perception, situation assessment and decisions about the actions to be taken. It also provides a mechanism for representing memory characteristics of the aircrew. A fourth activity, within the model, evaluates the extent of any factors that may affect the performance of the aircrew (so-called 'affectors'). These include physical affectors such as aircraft vibration, g-forces, temperature, cockpit pressure, and psychological affectors such as fear, work overload, etc.

The behaviour and reasoning characteristics of the aircrew model, written in Fril, are accessed from an Ada simulation framework via C data structures and control routines. For example, the data perception task determines what instruments or other data the aircrew would examine as a result of an event, and obtains that data, if available, from the simulation framework. This perceived data is passed on to the situation assessment task which determines the state of the aircraft within its current context and its ability to fulfil its mission. Using this perceived situation, the aircrew model decides what actions should be performed to optimise the success of the mission, addressing any outstanding problems.

Fril lends itself ideally to the modelling of such a task by its ability to combine rule-based reasoning with the qualitative analysis capabilities afforded by support logic and fuzzy sets. The uncertainty handling mechanism proves particularly useful in the evaluation of affectors and in representing the degree to which the resultant effects should be applied to the conscious tasks of the aircrew classification and case-based reasoning.

5.5 *Orthodontic expert system*

An expert system for planning orthodontic treatment [35, 36] was originally developed at the University of Bristol in collaboration with practitioners at the Bristol Dental Hospital. Recently this work was extended and the system is commercially available from Team Management Systems Ltd [37].

Orthodontics concerns the management of growth and development of teeth to prevent or correct such problems as overcrowding or misalignment. The development of the system involved analysing the decision-making processes used by orthodontists, to identify possible methods of emulating the process by computer. This resulted in a knowledge representation which used fuzzy sets since

orthodontic reasoning routinely involves the manipulation of vague or uncertain concepts.

The expert system has been tested extensively using a number of general dental practitioners entering a sample of cases. The system's recommendations have been shown to be comparable to the recommendations of a consultant orthodontist when assessed by peer review in a blind trial.

5.6 *Design/assessment of composite materials*

CODEX [38] is an expert system for the design and assessment of composite plates and struts, being developed by the University of Bristol and the Civil Aircraft Division of British Aerospace.

Due to the orthotropic macro-mechanical behaviour of advanced fibre-reinforced composites, layers of material are bonded with fibres in different orientations to give all-round strength. Optimisation of the amount of material with respect to weight, cost, stiffness, strength and other less quantifiable factors is central. The computer system aids users in the design of plates and struts, and allows assessment of the relative merits of previous designs based on such criteria. The plate or strut designs obtained can be compared by using a set of rules representing different design criteria. The relative importance of the rules can be set by the user, and an overall assessment for a design is derived using Fril.

5.7 *Administration of money market services*

CAMES (Client Administration Expert System) uses conceptual graphs for knowledge representation [39]. This application concerns the administration of money market services and was developed for commercial exploitation by Reuters Ltd in 1989. Reuters developed the system in Fril using a conceptual graph tool-kit provided by Fril Systems Ltd.

6. **Other fuzzy and possibility-based approaches**

This section briefly mentions some other systems which have been implemented for knowledge-based systems under uncertainty — ignoring the numerous proposals for systems to deal with uncertainty in knowledge-based systems, and concentrating only on those which have been implemented as practical, usable systems. The focus is on logic programming and related systems, ignoring the so-called fuzzy development environments which are targeted on simple fuzzy control applications. Although it can be argued that a fuzzy controller is a knowledge-based system — it has a rule-base, an inference procedure, and mechanisms for handling data fusion and conflict — such systems are little more than sophisticated interpolation algorithms, and cannot easily deal with issues such as:

- hierarchical and recursive rules,
- explanation of results,
- theoretical understanding of the results.

In addition to the logic programming systems, brief comments on production systems extended with fuzzy capabilities are included. These are similar in capability and scope of application to the logic programming formalism. A major disadvantage compared to logic programming is the complex control strategy and the difficulty in defining recursive rules — while not insurmountable, this is considerably less elegant than the natural way in which recursion can be expressed and controlled in a Prolog-like system.

6.1 *FProlog*

Fril version 1 was developed in the late 1970s following Baldwin's work on fuzzy relations, and version 2 followed in the early 1980s. Both were true relational languages. In order to add a procedural capability, a fuzzy Prolog interpreter [40] was written and integrated with the relational system. The interpreter could also function as a stand-alone fuzzy Prolog system. It allowed fuzzy relations to be defined, with a membership value giving the degree of truth of the tuple. Standard fuzzy operators (such as min/max) were used to compute truth values for conjunctions, etc, and to combine truth values arising from different proof paths for the same conclusion. Uncertainty was not allowed for rules, although it was possible to circumvent this by programming appropriate calculations. Uncertainty in attribute values was also modelled by programming rather than as a fundamental part of the language — hence no modification to the unification algorithm was needed. Recursive rules were possible, although as no uncertainty was attached to rules this adds little to the uncertainty modelling aspects of the language.

6.2 *Fuzzy Prolog*

Mukaidono has worked on fuzzy logic programming for a considerable time [41, 42], his most recent contribution being a fuzzy Prolog based on Lukasiewicz implication (LbFP), summarised in Martin and Arcelli Fontana [43]. In LbFP, uncertain attribute values are not allowed as fundamental objects, although it is possible to simulate their effect using programming techniques. No changes to the unification algorithm are therefore necessary. Facts and rules are augmented with truth values in the interval $[0, 1]$ where 1 represents truth and 0 represents unknown or meaningless (this is related to standard Prolog where failure to prove is only equivalent to proof of falsity under the assumptions of negation by failure). The truth values with a rule body are combined using min/max/1 - x for conjunction, disjunction, and negation respectively. The max operator is also used to combine truth values from different proof paths

for the same solution. LbFP is formulated in terms of tree resolution rather than the more usual linear resolution.

Mukaidono and Yasui [42] consider a number of implication operators, concluding that Lukasiewicz implication is the most appropriate for their needs. In particular, it can be used for recursive rules such as:

$X+1$ is a small number if X is a small number (with some degree of truth $m < 1$)

Given the base case that 0 is definitely a small number, Mukaidono and Yasui require that ∞ has membership zero in the set of small numbers defined by the extension of this rule. They note that many candidate implication operators can be ruled out because they do not reach zero in this limit — indeed, some do not decrease at all.

6.3 *f-PROLOG*

Li and Liu [44] defined a fuzzy Prolog in a manner reminiscent of the FProlog system discussed above. Their extensions were:

- in allowing uncertain rules, $P : -[f] - Q_1, Q_2, \dots, Q_n$ where P, Q_i are logic atoms and the conclusion P holds with the product of f and the minimum truth value associated with the body goals, Q_i ,
- in specifying a minimum threshold for a query to be satisfied, e.g. find a solution with truth value > 0.8 .

6.4 *Prolog-ELF*

Prolog-ELF [45] was a product of the early Fifth Generation research programme. It augmented clauses with a truth value in the interval $[0, 1]$, which were either assigned at compile time or (possibly) computed at run-time. The system is based on Lee's fuzzy resolution method [17], and clauses are written in the form:

$$+ P - Q - R$$

to emphasise that resolution is being used. The standard min/max/1 - x operations are used for conjunction, disjunction, and negation; in line with Lee, truth values of positive literals are required to be in the interval $(0.5, 1]$. Multiple proof paths are combined using max, and no uncertainty is allowed in attribute values.

6.5 *Fuzzy production systems*

In many respects, fuzzy production systems are close to fuzzy logic programming systems as they use rules augmented by uncertainty, and may also allow uncertain data values. The main difference is one of interpretation — in production systems, rules are interpreted only

procedurally, as condition/action pairs. Production systems may adopt forward or backward chaining execution mechanisms, and generally examine all rules rather than the fixed search strategy usually found in logic programming. This may be a disadvantage when predictability of program behaviour is considered.

FLOPS [46]¹ is a fuzzy extension to the OPS-5 production system. Data values may be discrete or continuous fuzzy sets. Continuous fuzzy sets must have Gaussian membership functions, and can model fuzzy numbers including arithmetic operations. Rules and facts are qualified by 'confidence values' which are integers in the range 0—1000; a min-max calculus is used to compute confidence values of results.

FuzzyCLIPS [47] is an extension of CLIPS [48] for representing and maintaining fuzzy facts and rules. CLIPS is a forward-chaining rule-based language that is similar to OPS5 in terms of its inferencing and representation capabilities. CLIPS is an expert system tool written in and fully integrated with the C language which makes it highly portable, relatively low in cost and can be easily integrated with external systems. Its current implementation supports many software paradigms in a single platform, including object-oriented, rule-based and procedural programming. FuzzyCLIPS supports both fuzziness and uncertainty. Fuzziness is defined using the membership function representation of facts with extensions for linguistic hedges. Uncertainty is supported using the concept of certainty factors that can take values in the range 0 to 1. These can be used for rules including crisp or fuzzy terms.

7. Benchmarking problems

In this section are listed some simple problems to illustrate different capabilities of knowledge-based systems involving uncertainty. The neural net and machine learning communities are already well advanced on this route with a number of Web sites (see <http://www.ics.uci.edu/~mllearn/MLRepository.html> and links contained there); similarly, there are several well-known fuzzy control problems such as the inverted pendulum, truck reversal, etc. The list concentrates on knowledge-based systems; it is not intended to be comprehensive, but is a starting point to stimulate further contributions. The problems are not intended to be computationally difficult, but rather to illustrate the concepts which can be expressed within a particular framework or system.

- Fuzziness and probability

You are told that a fair dice is thrown and the value is **small** where **small** is a fuzzy set defined as:

small = 1 : 1 + 2 : 0.9 + 3 : 0.4 where the notation element : membership is used.

Can the distribution $Pr(\text{dice is } i \mid \text{dice is } \textit{small})$ be derived?

What if the dice is weighted, with prior:

1 / 0.1, 2 / 0.2, 3 / 0.3, 4 / 0.2, 5 / 0.1, 6 / 0.1 where the notation element / probability is used.

What is $Pr(\text{dice is } \textit{about_2} \mid \text{dice is } \textit{small})$ where **about_2** is a fuzzy set defined as:

about_2 = 1 : 0.3 + 2 : 1 + 3 : 0.3

(see Baldwin and Martin [49] for details of the mass assignment solution to this problem).

- Probability, fuzziness and arithmetic

— Two fair dice are thrown, the value of dice 1 is **small** (defined above) and the value of dice 2 is **large**, where **large** is:

large = 4 : 0.3 + 5 : 1 + 6 : 1

What is the total score?

— If dice 1 is biased as above, what is the total?

— John is of **average height** and Bill is several centimetres taller than John (with suitable definitions for **average height** and **several** centimetres). What is the height of Bill? Is he **tall** (given a suitable definition of tall)?

— Take three approximate values given by the continuous fuzzy subsets of the real line :

$F1 = [-1:0 \ 1:1 \ 3:0]$,

$F2 = [2:0 \ 3:1 \ 4:0]$, and

$F3 = [1:0 \ 2:1 \ 3:0]$,

with linear interpolation between the points. Calculate the following quantities:

the sum $F1 + F3$, and difference $F1 - F3$

the product $F1 * F3$,

$F2 * (F1 + F3)$

$(F2 * F1) + (F2 * F3)$

$(F1 - F3)^2$

- Some simple database problems

— Take a small database of Name $\times$ Nationality $\times$ Height — say 200 tuples, almost all having Nationality = Swedish and height satisfying the definition of **tall** from the previous example. From this database, is it true so say that most Swedes are tall?

176 ¹ See also <http://users.aol.com/fuzzysys/>

Throw away the data and replace it with the summary 'Most Swedes are tall'. From this summary, what is the average height of Swedes? See also [5]).

— Take a small database of Name $\times$ Subject $\times$ Exam mark. Find the names of good project students, where a good project student is defined as one with mostly good exam scores, at least a few excellent marks, and very few or no poor marks. Appropriate definitions to be used:

'good', 'excellent', and 'poor' on the domain of exam marks

'mostly', 'at least a few', and 'very few or no' on the domain of proportions [0, 1].

- Machine learning/knowledge discovery in databases

There are several collections of datasets suitable for testing software which implements some form of learning. The majority of this data is numerical and consists of relatively large datasets, i.e. a size suitable for statistical treatment as well as neural net and other machine learning approaches. It would be useful to extend this body of data in two ways:

— to include symbolic data as well as numeric,

— to test generalisation from small data sets as well as interpolative reasoning in large datasets.

As an example, consider the following problem from Baldwin [50]: a mask is randomly dropped on to the grid shown in Fig 2 and moved until at least one pixel is found. Patterns for good L and bad L are shown. Not all of these patterns are discriminatory, and some occur twice as frequently as the corresponding pattern in the other classification. It is known that errors in transmission can occur so that the third row of patterns (unknown classification) can occur; it is assumed that only one pixel is affected in the case of an error. Given any one of the patterns, classify the object as a good L or a bad L .

Clearly the list above is a small sample of the total required. Other areas that should be addressed include:

- sensor processing — pattern recognition, vision, speech/sound recognition, olfactory sensors,

- fuzzy control problems — inverted pendulum, truck reversal, air conditioning,
- preprocessing data — clustering, classifying, removal of noise,
- longer inference chains involving uncertain rules,
- data fusion — how to combine different sources and resolve conflicts.

8. Conclusions

Knowledge-based systems involving uncertainty should be able to provide a semantics for their representation of uncertainty in data and inference. As Bezdek [51] said, we should "...think about how we use terms such as intelligent system" and "...discourage the use of seductive semantics in algorithmic descriptions and encourage the use of strict verifiable definitions of computational properties." Some simple problems have been outlined as demonstrators for the capabilities of knowledge-based systems using soft computing techniques. Collaboration from software providers (both commercial and academic) and researchers in a range of application areas will be needed to make progress towards a complete benchmarking suite for the capabilities of soft computing tools.

It is perhaps worth emphasising that the early expert system success stories such as Mycin and Prospector included treatment of uncertainty, and arguably succeeded because of this. The role of uncertainty in knowledge-based systems is crucial, and logic programming is one of the main approaches to actually building knowledge-based systems, as opposed to discussing and philosophising about them. This paper has outlined the extensions to logic programming necessary to handle uncertainty, concentrating on the practical rather than theoretical aspects. In particular, the Fril approach to the problem has been discussed, together with some other fuzzy Prolog systems.

There is still work to do in the field, but the combination of logical inference and uncertainty management techniques have much to offer in the implementation of intelligent information systems.

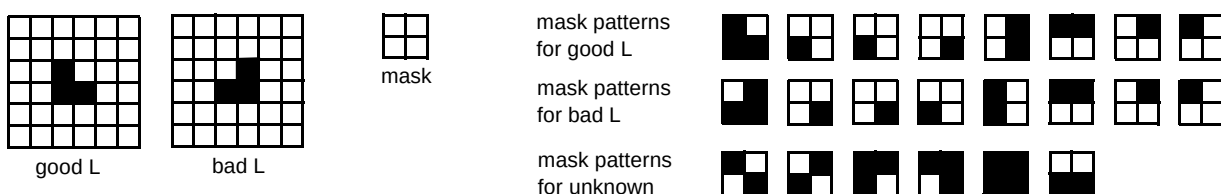


Fig 2 Mask problem [50].

References

- 1 Zadeh L A: 'The roles of fuzzy logic and soft computing in the conception, design and deployment of intelligent systems', in Nwana H S and Azarmi N (Eds): 'Software agents and soft computing: concepts and applications', Springer Verlag, pp 183—190 (1997).
- 2 Baldwin J F and Martin T P: 'A fuzzy data browser in Fril', in Baldwin J F (Ed): 'Fuzzy logic in AI', John Wiley, pp 101—124 (1996).
- 3 Baldwin J F and Martin T P: 'From fuzzy databases to an intelligent manual using Fril', *Journal of Intelligent Information Systems*, 2, pp 365—395 (1993).
- 4 Baldwin J F and Martin T P: 'The intelligent manual in Fril', *Proc EUFIT93 — First European Congress on Fuzzy and Intelligent Technologies*, pp 624—630, Aachen, Germany (1993).
- 5 Zadeh L A: 'Fuzzy logic — computing with words', *IEEE Transactions on Fuzzy Systems*, 4, pp 103—111 (1996).
- 6 Dubois D, Prade H and Yager R R: 'Information engineering and fuzzy logic', *Proc FUZZ-IEEE 96*, pp 1525—1531 (1996).
- 7 Baldwin J F: 'A calculus for mass assignments in evidential reasoning', in Fedrizzi M, Kacprzyk J and Yager R R (Eds): 'Advances in the Dempster-Shafer theory of evidence', John Wiley (1992).
- 8 Baldwin J F: 'The management of fuzzy and probabilistic uncertainties for knowledge based systems', in Shapiro S A (Ed): 'Encyclopedia of AI', John Wiley (2nd edition), pp 528—537 (1992).
- 9 Bezdek J C: 'Fuzzy models', *IEEE, Trans Fuzzy Systems*, 1, No 1, pp 1—5 (1993).
- 10 Baldwin J F, Martin T P and Pilsworth B W: 'Fril — Fuzzy and evidential reasoning in AI', *Research Studies Press (John Wiley)* (1995).
- 11 Dubois D and Prade H: 'Fuzzy sets in approximate reasoning 1 — inference with possibility distributions', *Fuzzy Sets and Systems*, 40, pp 143—202 (1991).
- 12 Yager R R: 'On different classes of linguistic variables defined via fuzzy subsets', *Kybernetes*, 13, pp 103—110 (1984).
- 13 Kowalski R: 'Logic for problem solving', Elsevier (1979).
- 14 Robinson J A: 'A machine oriented logic based on the resolution principle', *JACM*, 12, pp 23—41 (1965).
- 15 Virtanen H E: 'Linguistic logic programming', in Martin T P and Arcelli Fontana F (Eds): 'Logic programming and soft computing', pp 91—110, Wiley (1998).
- 16 Baldwin J F, Lawry J and Martin T P: 'Efficient algorithms for semantic unification', *Proc Information Processing and the Management of Uncertainty*, pp 527—532, Spain (1996).
- 17 Lee R C T: 'Fuzzy logic and the resolution principle', *JACM*, 19, pp 109—119 (1972).
- 18 Nguyen H T: 'On random sets and belief functions', *J Math Anal and Appl*, 65, pp 531—542 (1978).
- 19 Shafer G: 'A mathematical theory of evidence', Princeton University Press (1976).
- 20 Baldwin J F: 'Evidential reasoning under probabilistic and fuzzy uncertainties', in Yager R R and Zadeh L A (Eds): 'An introduction to fuzzy logic and applications in intelligent systems', Kluwer Academic Publishers, pp 297—335 (1991).
- 21 Baldwin J F: 'A new approach to inference under uncertainty for knowledge based systems', in Kruse R and Siegel P (Eds): 'Symbolic and quantitative approaches to uncertainty', Springer Verlag, pp 107—115 (1991).
- 22 Baldwin J F: 'Fuzzy sets, fuzzy clustering, and fuzzy rules in AI', in Ralescu A L (Ed): 'Fuzzy logic in AI', Springer Verlag, pp 10—23 (1993).
- 23 Baldwin J F and Martin T P: 'An abstract mechanism for handling uncertainty', in Bouchon-Meunier B, Yager R R and Zadeh L A (Eds): 'Uncertainty in knowledge bases', Springer Verlag, pp 126—135 (1991).
- 24 Warren D H D: 'An abstract Prolog instruction set', SRI International, Menlo Park, CA (1983).
- 25 Baldwin J F and Martin T P: 'An abstract mechanism for handling uncertainty', *Proc Information Processing and the Management of Uncertainty*, pp 333—336, Paris (1990).
- 26 Baldwin J F and Martin T P: 'Fast operations on fuzzy sets in the abstract Fril machine', *Proc First IEEE International Conference on Fuzzy Systems*, pp 803—810, San Diego, CA (1992).
- 27 Baldwin J F, Martin T P and Shanahan J G: 'Modelling with words using Cartesian granule features', *Proc FUZZ-IEEE-97*, pp 1295—1300, Barcelona, Spain (1997).
- 28 Baldwin J F, Martin T P and Stylianidis O: 'A soft computing approach to hand written numeral recognition', in Lazzerini B and Jain L (Eds): 'Intelligent Handwriting Recognition', CRC Press (1998).
- 29 Baldwin J F et al: 'Machine recognition of odours in Fril', in Baldwin J F (Ed): 'New applications in fuzzy logic', (to appear) (1998).
- 30 Baldwin J F and Martin T P: 'Fuzzy modelling in an intelligent data browser', *Proc 4th IEEE International Conference on Fuzzy Systems*, pp 1885—1890, Yokohama, Japan (1995).
- 31 Baldwin J F, Martin T P and McCoy S A: 'Incremental learning in a Fril-based odour classifier', *Proc EUFIT-98*, Aachen, Germany (1998).
- 32 Baldwin J F, Gooch R M and Martin T P: 'Fuzzy processing of hydrophone sounds', *Fuzzy Sets and Systems*, 77, No 1, pp 35—48 (1996).
- 33 SKI: 'Project-90', Technical Report 91, No 23, Statens Karnkraftinspektion, Stockholm, Sweden (1991).

- 34 Monk M R M and Swabey M: 'Simulation of aircrew behaviour for system integration using knowledge-based programming', *Proc Modelling and Simulation, ESM93*, pp 459—463, Lyons, France (1993).
- 35 Sims Williams J H, Brown I D and Mathewman A: 'An orthodontic expert system', *Fuzzy Sets and Systems*, 30, pp 121—133 (1989).
- 36 Sims Williams J H et al: 'A computer controlled expert system for orthodontic advice', *British Dental Journal*, 163, pp 161—166 (1987).
- 37 Mackin N and Stephens C D: 'Developing and testing a fuzzy expert system — an example in orthodontics', *Proc Fuzzy Logic — Applications and Future Directions*, pp 61—69, London (1997).
- 38 Wu C M L, Webber J P H and Morton S K: 'A knowledge-based expert system for laminated composite strut design', *Aeronautical Journal*, 95, pp 1—20 (1991).
- 39 Smith B: 'CAMES — expert system administration of money market services', *Proc Conceptual Graphs Workshop, European Conference on AI* (1990).
- 40 Baldwin J F, Martin T P and Pilsworth B W: 'The implementation of FProlog — a fuzzy prolog interpreter', *Fuzzy Sets and Systems*, 23, pp 119—129 (1987).
- 41 Mukaidono M, Shen Z L and Ding L: 'Fundamentals of fuzzy Prolog', *Int J Approximate Reasons*, 3, pp 179—193 (1989).
- 42 Mukaidono M and Yasui M: 'Postulates and proposals of fuzzy prolog', *Proc EUFIT 94*, pp 1080—1086, Germany (1994).
- 43 Martin T P and Arcelli Fontana F: 'Logic programming and soft computing', *Uncertainty in AI Research Studies Press*, John Wiley (1998).
- 44 Li D and Liu D: 'A fuzzy PROLOG database system', *Research Studies Press (John Wiley)* (1990).
- 45 Ishizuka M and Kanai N: 'PROLOG-ELF incorporating fuzzy logic', *New Generation Computing*, 4, pp 479—486 (1985).
- 46 Buckley J J, Siler W and Tucker D: 'FLOPS — a fuzzy expert system', *Fuzzy Sets and Systems*, 20, pp 1—16 (1986).
- 47 Orchard R A: 'FuzzyCLIPS user guide', *Knowledge Systems Lab, National Research Council, Canada* (1995).
- 48 Giarratano J C: 'CLIPS user's guide', *NASA Technical Report, Lyndon B Johnson Center* (1993).

49 Baldwin J F and Martin T P: 'Basic concepts of a fuzzy logic data browser with applications', in Nwana H S and Azarmi N (Eds): 'Software agents and soft computing: concepts and applications', Springer Verlag, pp 211—241 (1997).

50 Baldwin J F: 'Knowledge from data using Fril and fuzzy methods', in Baldwin J F (Ed): 'Fuzzy logic in AI', John Wiley, pp 33—76 (1996).

51 Bezdek J C: 'What is computational intelligence?', in Zurada J M, Marks R J (II) and Robinson C J (Eds): 'Computational intelligence — imitating life', IEEE Press, pp 1—12 (1994).



Jim Baldwin is Professor of Artificial Intelligence and Director of the AI Research Group at the University of Bristol. He has been active in fuzzy sets since the early days of the subject, and his research in AI covers fundamental aspects of knowledge representation, inference under uncertainty and belief theories, fuzzy control and fuzzy set theory, and machine learning. He is the originator of support logic programming and the mass assignment theory which was developed during his EPSRC Advanced Fellowship (1990—95). He has published over 250 papers, is a member of the editorial

board for a number of journals and has served on many conference program committees.



Trevor Martin is a Senior Lecturer in the AI Research Group at the University of Bristol. His research interests focus on uncertainty in AI, and he is co-developer of the Fril language and the fuzzy data browser.

He has published over 90 papers in refereed journals and conferences, and is a joint organiser of several international fuzzy and fuzzy logic programming workshops.



Behnam Azvine holds a BSc in Mechanical Engineering, an MSc and PhD in Control Systems from Manchester University. His previous work experience includes Research Associate and Lecturer in the Dynamics and Control Research group of the Engineering Department of Manchester University working on theoretical control, robust stability and smart system design, and Senior Lecturer in the Control and Instrumentation Research group of Staffordshire University working on smart structures and robotic control systems.

He is currently leading a research team responsible for soft computing and computational intelligence techniques in the Intelligent Systems Research group at BT Laboratories. He regularly gives presentations at international conferences and workshops on the application of AI in Telecommunications, and currently holds a visiting fellowship at the Department of Engineering Mathematics of Bristol University. He is also one of the chairmen of the European Network of Excellence for Uncertainty Management Techniques (ERUDIT). His current research interests include the application of fuzzy logic, neural networks and evolutionary computing to human-centred computing, intelligent system design, model-based control and adaptive software systems.